

Maison Tislit

E-COMMERCE PLATFORM

Projet de Fin d'Etudes

Full-Stack Web Development

2025

General Introduction

The Moroccan traditional clothing industry, known for its rich heritage of djellabas, takchitas, and gandouras, has long been rooted in local craftsmanship and physical marketplaces. In recent years, however, the global shift toward e-commerce has created new opportunities for artisanal brands to reach a wider audience. Yet, many Moroccan clothing brands still lack a dedicated digital presence that reflects the elegance and cultural significance of their products.

Maison Tislit was founded with the mission of celebrating Moroccan feminine clothing by blending traditional craftsmanship with contemporary design. To support this vision, we identified the need for a custom e-commerce platform — one that not only allows customers to browse and purchase products but also provides a seamless, multilingual, and culturally resonant shopping experience.

This project focuses on the design and development of a full-stack e-commerce website for Maison Tislit. The platform includes user authentication, a product catalog with filtering and sorting, a shopping cart, secure payment processing via Stripe, order management, and an administrative dashboard for managing products, orders, and customer messages. The application is built using React with TypeScript for the frontend and Supabase — a backend-as-a-service platform — for the database, authentication, and API layer.

This report is structured as follows: Chapter 1 presents a state-of-the-art analysis of the e-commerce landscape for traditional clothing and defines the problem statement. Chapter 2 covers the analysis and design phases, including functional requirements and system architecture. Chapter 3 details the technologies and tools selected for the project. Chapter 4 describes the implementation of each major feature. Chapter 5 discusses testing and validation. Finally, the conclusion summarizes the achievements and outlines future perspectives.

Chapter 1: State of the Art & Problem Statement

1.1 The Moroccan Traditional Clothing Market

Moroccan traditional clothing — including the djellaba, takchita, gandoura, and caftan — represents centuries of artisanal heritage. These garments are characterized by intricate embroidery, high-quality fabrics such as silk and velvet, and designs that vary by region. The city of Fez, where Maison Tislit is rooted, remains one of the most important centers for traditional textile craftsmanship.

Despite this rich tradition, the market faces several challenges:

- **Limited online presence:** Most traditional clothing sellers operate through physical stores or social media platforms like Instagram and WhatsApp, lacking a structured e-commerce experience.
- **Fragmented customer journey:** Customers often interact across multiple channels (Instagram for discovery, WhatsApp for inquiries, bank transfers for payment) resulting in a disjointed experience.
- **Language barriers:** Many existing platforms cater primarily to French or Arabic speakers, leaving English-speaking customers underserved.

1.2 Existing E-commerce Solutions

Several platforms exist in the Moroccan e-commerce space:

Solution	Type	Strengths	Limitations
Hespax	General marketplace	Large audience	Not specialized for traditional clothing
Avito.ma	Classified ads	High traffic	No integrated payment or cart system
Instagram/Facebook Shop	Social commerce	Easy setup	Limited customization, no proper cart/checkout
Shopify/WooCommerce	DIY platform	Feature-rich	Generic templates, monthly fees, limited tailoring

None of these solutions offer a dedicated, culturally tailored experience for a Moroccan traditional clothing brand with built-in multilingual support, an integrated admin dashboard, and a modern UI.

1.3 Problem Statement

Maison Tislit requires a custom-built e-commerce platform that:

1. Showcases traditional Moroccan clothing in a way that reflects its elegance and cultural value.
2. Provides a seamless shopping experience across French, English, and Arabic.
3. Integrates secure online payments via Stripe.
4. Includes an administrative interface for managing products, orders, and customer communications.
5. Supports user accounts with order history and favorites.
6. Delivers all functionality without recurring platform fees or restrictive templates.

1.4 Objectives

Primary Objectives

- Design and develop a full-stack e-commerce website for Maison Tislit.
- Implement a responsive, modern UI that reflects the brand's identity.

- Build a secure authentication and user management system.
- Create a product catalog with categories, filtering, and sorting.
- Develop a shopping cart and checkout flow with Stripe payment integration.
- Build an admin dashboard for product and order management.

Secondary Objectives

- Implement bilingual support (French and English) with Arabic-ready architecture.
- Add loyalty features such as favorites and order history.
- Ensure mobile responsiveness and accessibility.
- Deploy the application to a production environment.

1.5 Methodology

The project follows an iterative development approach:

1. **Requirements gathering:** Defining functional and non-functional requirements based on the brand's needs.
2. **Design:** System architecture, database schema, and UI/UX mockups.
3. **Implementation:** Building the frontend and backend components iteratively.
4. **Testing:** Verifying functionality, performance, and security.
5. **Deployment:** Deploying the application for production use.

1.6 Scope

The platform covers the following features:

- **Frontend:** Landing page, shop page, product detail modal, shopping cart, checkout flow, user account page, login/registration pages, and admin panel.
- **Backend:** PostgreSQL database, RESTful API via Supabase, authentication, and Stripe payment integration.
- **Admin:** Product CRUD operations, order management, messaging system, and user management.

Features explicitly out of scope for this version include: mobile native applications, AI-powered recommendations, and a reviews/ratings system.

Chapter 2: Analysis & Design

2.1 Functional Requirements

2.1.1 User-Facing Features

ID	Requirement	Description
FR1	Browse Products	Users can view all products with category filtering and sorting
FR2	Product Details	Users can view product images, descriptions, colors, sizes, and prices
FR3	User Registration	New users can create an account with name, email, and password
FR4	User Login	Registered users can log in securely
FR5	Shopping Cart	Users can add/remove products and update quantities
FR6	Checkout	Users can place orders with delivery information
FR7	Payment	Users can pay via credit/debit card through Stripe
FR8	Order Confirmation	Users receive an order confirmation with details
FR9	Account Management	Users can view order history and account information
FR10	Multilingual	The interface supports French and English

2.1.2 Admin Features

ID	Requirement	Description
FR11	Product Management	Admin can add, edit, and delete products
FR12	Order Management	Admin can view and update order statuses

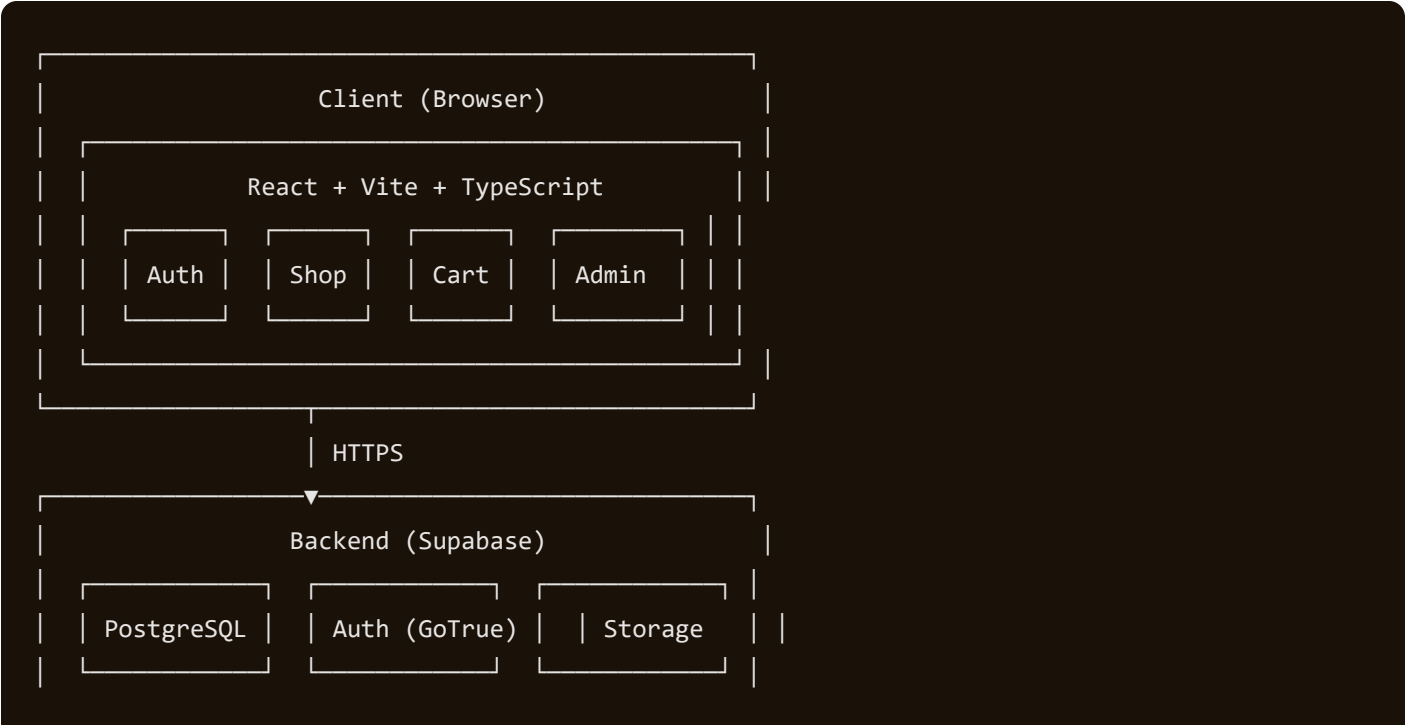
ID	Requirement	Description
FR13	Customer Messages	Admin can view and reply to customer inquiries
FR14	User Management	Admin can view registered users
FR15	Dashboard	Admin sees overview statistics (products, orders, revenue)

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR1	Security	Passwords hashed, authentication via Supabase, admin-only routes
NFR2	Responsiveness	Fully functional on desktop, tablet, and mobile
NFR3	Performance	Fast page loads, optimized images, lazy loading
NFR4	Scalability	Backend scales via Supabase managed infrastructure
NFR5	Maintainability	Clean code structure, TypeScript for type safety

2.3 System Architecture

The application follows a modern full-stack architecture:





2.3.1 Frontend Architecture

- **Framework:** React 18 with TypeScript
- **Build Tool:** Vite for fast development and optimized builds
- **Styling:** Tailwind CSS for utility-first responsive design
- **Routing:** React Router v6 for client-side navigation
- **State Management:** React Context API for global state (auth, cart, language)
- **Icons:** Lucide React for consistent iconography

2.3.2 Backend Architecture

- **Database:** PostgreSQL (managed by Supabase)
- **Authentication:** Supabase Auth (GoTrue) with JWT tokens
- **API:** Direct Supabase client queries from the browser (Row Level Security)
- **Payments:** Stripe via a Supabase Edge Function (create-payment-intent)
- **File Storage:** Supabase Storage for product images (future use)

2.4 Database Schema

The database consists of the following tables:

profiles

Column	Type	Description
id	UUID	Primary key, references auth.users
name	TEXT	User's full name

Column	Type	Description
email	TEXT	User's email address
is_admin	BOOLEAN	Admin flag
created_at	TIMESTAMP	Account creation date

products

Column	Type	Description
id	BIGINT	Primary key, auto-increment
name	TEXT	Product name (French)
name_en	TEXT	Product name (English)
name_ar	TEXT	Product name (Arabic)
category	TEXT	Category (djellaba, takchita, gandoura)
price	NUMERIC	Current price
original_price	NUMERIC	Original price (for discounts)
image	TEXT	Image URL path
description	TEXT	Description (French)
description_en	TEXT	Description (English)
sizes	TEXT[]	Available sizes
colors	TEXT[]	Available colors
badge	TEXT	Badge (Bestseller, Nouveau, etc.)
stock	INTEGER	Stock quantity
created_at	TIMESTAMP	Product creation date

orders

Column	Type	Description
id	BIGINT	Primary key, auto-increment
user_id	UUID	References profiles.id
status	TEXT	pending, shipped, delivered
total	NUMERIC	Order total
address	TEXT	Shipping address
city	TEXT	Shipping city
phone	TEXT	Customer phone
created_at	TIMESTAMP	Order date

order_items

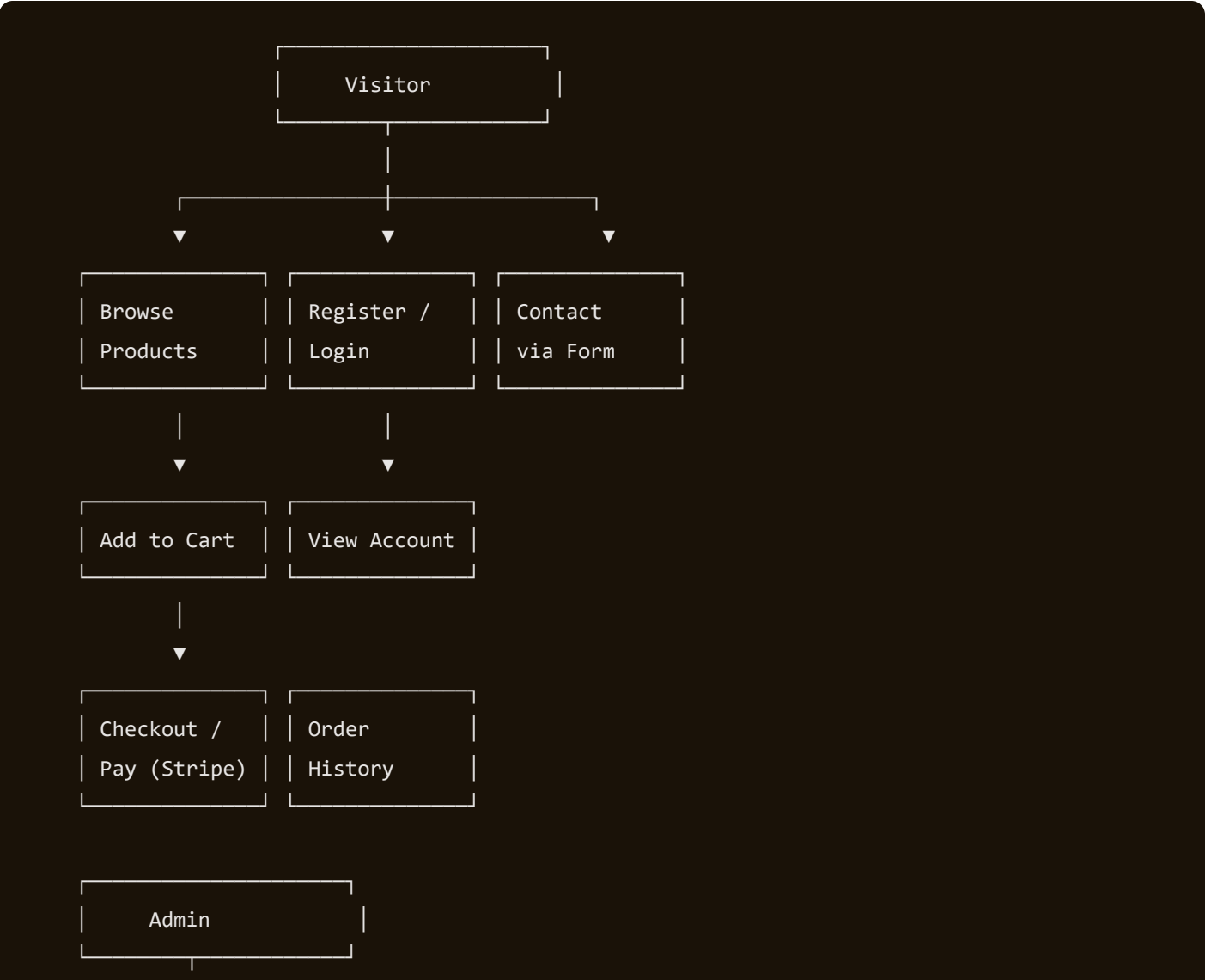
Column	Type	Description
id	BIGINT	Primary key
order_id	BIGINT	References orders.id
product_id	BIGINT	References products.id
product_name	TEXT	Snapshot of product name
quantity	INTEGER	Quantity ordered
price	NUMERIC	Price at time of order
size	TEXT	Selected size
color	TEXT	Selected color

messages

Column	Type	Description
id	BIGINT	Primary key

Column	Type	Description
name	TEXT	Sender name
email	TEXT	Sender email
subject	TEXT	Message subject
body	TEXT	Message content
is_read	BOOLEAN	Read status
reply	TEXT	Admin reply
created_at	TIMESTAMP	Message date

2.5 Use Case Diagram





2.6 UI/UX Design

2.6.1 Design System

- **Color Palette:**
 - Primary: Stone (#1a1208, #44403c)
 - Accent: Amber (#d97706, #c9a84c)
 - Background: Stone-50 (#fafaf9), White (#ffffff)
 - Text: Stone-900 (#1c1917), Stone-400 (#a8a29e)
- **Typography:**
 - Headings: Playfair Display (serif, elegant)
 - Body: Raleway (sans-serif, modern)
 - Accent: Cormorant Garamond (serif, refined)
- **Component Patterns:** Rounded cards with subtle shadows, hover animations, clean grid layouts.

2.6.2 Page Structure

1. **Home Page:** Hero video, story section, category cards, featured products, brand values
2. **Shop Page:** Filter bar, product grid with cards, sort options
3. **Cart Page:** Item list, quantity controls, order summary
4. **Checkout Page:** Multi-step form (delivery → payment)
5. **Account Page:** Order history, account details
6. **Admin Page:** Sidebar navigation, tabbed sections (overview, stock, orders, messages, users)

Chapter 3: Technologies & Tools

3.1 Frontend Stack

3.1.1 React

React is a JavaScript library for building user interfaces, developed by Meta. It was chosen for its component-based architecture, which promotes reusability and maintainability. React's virtual DOM ensures efficient rendering, making it suitable for a dynamic e-commerce interface.

Key advantages for this project:

- Component reuse (ProductCard, Badge, Modal components)
- Efficient state management via Context API
- Large ecosystem and community support

3.1.2 TypeScript

TypeScript adds static type checking to JavaScript, catching errors at compile time rather than runtime. This is particularly valuable for an e-commerce application where data structures (products, orders, cart items) must be consistent across the application.

Key types defined:

- `Product` : name, price, category, sizes, colors, image
- `CartItem` : extends Product with quantity, selectedSize, selectedColor
- `User` : id, name, email
- `Page` : union type for routing

3.1.3 Vite

Vite is a modern build tool that offers fast development server startup via native ES module serving and optimized production builds via Rollup. It was chosen over Create React App for its significantly faster hot module replacement (HMR) and smaller configuration overhead.

3.1.4 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that enables rapid UI development through pre-built classes. It was selected for:

- **Consistency:** Predefined design tokens for colors, spacing, typography.
- **Responsiveness:** Built-in breakpoint prefixes (`sm:` , `md:` , `lg:`).
- **Customization:** The `tailwind.config.js` allows extending the default theme.
- **Performance:** Purges unused CSS in production builds.

3.2 Backend Stack

3.2.1 Supabase

Supabase is an open-source backend-as-a-service (BaaS) that provides a PostgreSQL database, authentication, storage, and Edge Functions. It was chosen for several reasons:

PostgreSQL Database:

- Full relational database with SQL queries, joins, and migrations.
- Row Level Security (RLS) for fine-grained access control.
- Real-time subscriptions (future scalability).

Authentication (GoTrue):

- Built-in user management with email/password authentication.
- JWT-based session handling.
- Integration with React via `@supabase/supabase-js` .

Edge Functions:

- Deno-based serverless functions for secure operations like Stripe payment intent creation.
- Deployed and managed through Supabase CLI.

Why Supabase over alternatives:

Feature	Supabase	Firebase	Custom Backend
Database	PostgreSQL (SQL)	Firestore (NoSQL)	Any

Feature	Supabase	Firebase	Custom Backend
Auth	Built-in	Built-in	Manual
Pricing	Free tier generous	Pay-as-you-go	Server costs
Open Source	Yes	No	N/A
Real-time	Yes	Yes	Manual

3.2.2 Stripe

Stripe provides the payment processing infrastructure. The integration uses:

- **Stripe Elements:** Secure payment form components.
- **Payment Intent API:** Creates a payment intent server-side via a Supabase Edge Function.
- **Confirm Card Payment:** Client-side confirmation with Stripe.js.

The payment flow is as follows:

1. Client sends cart total to Edge Function
2. Edge Function creates a PaymentIntent via Stripe API
3. Client receives `client_secret`
4. Client confirms payment with card details via Stripe Elements
5. Stripe processes the payment and returns a result
6. On success, order is created in the database

3.3 Development Tools

Tool	Purpose
VS Code	Code editor
Git	Version control
npm	Package management
Supabase CLI	Local development, migrations, function deployment
Chrome DevTools	Debugging and performance analysis

3.4 Deployment

The application is deployed as a static site (since Supabase handles the backend):

- **Frontend:** Built with `vite build` and deployed to a hosting service (e.g., Vercel, Netlify, or Supabase Hosting).
- **Backend:** Fully managed by Supabase cloud.
- **Environment Variables:** Supabase URL and anon key configured in the hosting platform.

Chapter 4: Implementation

4.1 Project Setup and Structure

The project was initialized using Vite with the React + TypeScript template. The directory structure is organized as follows:

```
maison-tislit-e-commerce-website/
├── public/
│   └── images/          # Static assets (product images, HERO.mp4)
├── src/
│   ├── components/      # Reusable UI components
│   │   ├── Footer.tsx
│   │   ├── Navbar.tsx
│   │   ├── ProductModal.tsx
│   │   ├── Toast.tsx
│   │   └── WhatsAppButton.tsx
│   ├── context/         # React Context providers
│   │   ├── AppContext.tsx # Global state (auth, cart, products)
│   │   └── LanguageContext.tsx # i18n state management
│   ├── pages/           # Page-level components
│   │   ├── HomePage.tsx
│   │   ├── ShopPage.tsx
│   │   ├── CartPage.tsx
│   │   ├── CheckoutPage.tsx
│   │   ├── ConfirmationPage.tsx
│   │   ├── LoginPage.tsx
│   │   └── RegisterPage.tsx
```

```

| | |─ AccountPage.tsx
| | |─ AdminPage.tsx
| |─ supabase/      # Supabase integration
| | |─ client.ts    # Supabase client initialization
| | |─ queries.ts   # Database query functions
| | |─ types.ts     # Database row type definitions
| |─ translations/  # i18n files
| | |─ fr.ts        # French translations
| | |─ en.ts        # English translations
| |─ utils/
| | |─ cn.ts        # Tailwind class merging utility
| |─ App.tsx        # Root component with routing
| |─ main.tsx       # Application entry point
| |─ types.ts       # Shared TypeScript interfaces
| |─ index.css      # Tailwind directives + global styles
|─ supabase/
| |─ migrations/    # Database migrations
| | |─ 00001_schema.sql
| |─ functions/     # Edge Functions
| | |─ create-payment-intent/
| | | |─ index.ts
|─ index.html
|─ vite.config.ts
|─ package.json

```

4.2 Authentication System

Authentication is handled by Supabase Auth and wrapped in the AppContext. The implementation covers three operations: login, registration, and logout.

Login flow:

```

// src/context/AppContext.tsx (simplified)
const login = async (email: string, password: string) => {
  const { data, error } = await supabase.auth.signInWithPassword({ email, password });
  if (error) return false;
  // Fetch user profile
  const profile = await getProfile(data.user.id);
  setUser({ id, name, email });
  setProfile(profile);
}

```

```
    return true;
  };
```

Row Level Security (RLS):

- Products are readable by anyone (`SELECT` policy for all users).
- Orders are only readable by the owning user or an admin.
- Admin routes and operations are protected by checking `is_admin` on the profiles table.

4.3 Product Catalog

Products are stored in the `products` PostgreSQL table and fetched via the `getProducts()` query function:

```
export async function getProducts(): Promise<ProductRow[]> {
  const { data, error } = await supabase
    .from('products')
    .select('*')
    .order('created_at', { ascending: false });
  return data || [];
}
```

4.3.1 Category Filtering

The ShopPage implements client-side filtering by category using React state:

```
const filtered = products
  .filter(p => activeCategory === 'all' || p.category === activeCategory)
  .sort((a, b) => {
    if (sortBy === 'price-asc') return a.price - b.price;
    if (sortBy === 'price-desc') return b.price - a.price;
    return 0;
  });
```

4.3.2 Product Cards

Each product is displayed as a `ProductCard` component with:

- Product image with hover zoom effect

- Badge display (Bestseller, Nouveau, Premium, Promo)
- Discount percentage when originalPrice is set
- Color chips showing available colors
- Quick-add-to-cart button

4.4 Shopping Cart

The cart is managed through React Context and persisted to localStorage:

```
const [cart, setCart] = useState<CartItem[]>(() => {  
  const saved = localStorage.getItem('maison-tislit-cart');  
  return saved ? JSON.parse(saved) : [];  
});
```

Operations:

- **Add to cart:** Checks for existing item with same ID, size, and color; increments quantity if found, otherwise adds new item.
- **Remove from cart:** Filters out the matching item.
- **Update quantity:** Changes quantity or removes if zero.
- **Cart persistence:** Cart is saved to localStorage on every change via a `useEffect`.

4.5 Checkout & Payment

4.5.1 Checkout Flow

The checkout is a two-step process:

1. **Delivery Information:** User fills in name, email, phone, address, and city.
2. **Payment:** User enters card details via Stripe Elements.

4.5.2 Stripe Integration

Server-side (Edge Function):

```
// supabase/functions/create-payment-intent/index.ts
const paymentIntent = await stripe.paymentIntents.create({
  amount: amount * 100, // Convert to cents
  currency: 'eur',
  automatic_payment_methods: { enabled: true },
});
```

Client-side:

```
const { error } = await stripe.confirmCardPayment(clientSecret, {
  payment_method: { card: elements.getElement(CardElement) },
});
```

4.5.3 Order Creation

On successful payment, an order is created in the database with:

- User ID, total amount, delivery information
- Order items with product snapshots (name, price, size, color at time of purchase)
- Status set to "pending"

4.6 Multilingual Support

The application supports French and English through a custom translation system:

```
// LanguageContext provides current locale and translation function
const { t, locale, setLocale } = useTranslation();

// Usage in components
<h1>{t('shop.ourBoutique')}</h1>
```

Translation files export key-value objects:

```
// fr.ts
const fr = {
  'nav.home': 'Accueil',
  'shop.ourBoutique': 'Notre Boutique',
  // ...
}
```

```
};

// en.ts
const en = {
  'nav.home': 'Home',
  'shop.ourBoutique': 'Our Boutique',
  // ...
};
```

Product names and descriptions are stored in both French and English in the database, and the UI selects the appropriate language based on the current locale.

4.7 Admin Dashboard

The admin panel is a protected page accessible only to users with `is_admin = true`. It features:

4.7.1 Overview Section

- Statistics cards: Total products, pending orders, unread messages, registered users, revenue.
- Data fetched from the database and displayed in a clean grid layout.

4.7.2 Stock Management

- Searchable, filterable product table.
- Add/edit modal with fields for all product attributes.
- Delete with confirmation.

4.7.3 Order Management

- Orders displayed in a table with status badges (pending/shipped/delivered).
- One-click status updates.
- Filters by status.

4.7.4 Message Management

- Customer inquiries displayed in a read/unread view.
- Reply functionality with inline form.

4.7.5 User Management

- Table of registered users with admin status indicator.
- Ability to toggle admin privileges.

4.8 Performance Optimizations

- **Lazy loading:** Images use `loading="lazy"` for deferred loading off-screen.
- **CSS:** Tailwind purges unused styles in production builds.
- **Build:** Vite produces optimized chunks with code splitting.

4.9 Key Screenshots

(Insert screenshots of: Homepage, Shop Page, Product Modal, Cart, Checkout, Admin Dashboard)

Chapter 5: Testing & Validation

5.1 Testing Strategy

Testing was performed at multiple levels to ensure the application functions correctly across all features. Given the project's scope, the testing strategy focused on:

- **Functional testing:** Verifying that each feature works as specified in the requirements.
- **Integration testing:** Ensuring frontend components work correctly with the Supabase backend.
- **Responsive testing:** Verifying the UI renders correctly on different screen sizes.
- **User acceptance testing:** Manual testing by simulating real user flows.

5.2 Functional Testing

5.2.1 Authentication

Test Case	Steps	Expected Result	Status
User registration	Fill registration form, submit	Account created, redirected	✓ Pass
User login	Enter credentials, submit	Logged in, toast notification	✓ Pass
Login with wrong password	Enter incorrect password	Error message displayed	✓ Pass
Logout	Click logout button	Session cleared, redirected to home	✓ Pass
Protected routes	Access /account while logged out	Redirected to login	✓ Pass

5.2.2 Product Browsing

Test Case	Steps	Expected Result	Status
View all products	Navigate to shop	All products displayed	✓ Pass
Filter by category	Click category button	Products filtered correctly	✓ Pass
Sort by price	Select sort option	Products sorted ascending/descending	✓ Pass
Product details	Click product card	Modal opens with details	✓ Pass

5.2.3 Cart Operations

Test Case	Steps	Expected Result	Status
Add to cart	Click add button, logged in	Item added, toast shown	✓ Pass
Add to cart (guest)	Click add button, not logged in	Modal opens for login prompt	✓ Pass
Update quantity	Change quantity in cart	Total updates correctly	✓ Pass
Remove item	Click remove	Item removed from cart	✓ Pass
Cart persistence	Refresh page	Cart items preserved	✓ Pass

5.2.4 Checkout & Payment

Test Case	Steps	Expected Result	Status
Fill delivery form	Enter valid info	Proceed to payment step	✓ Pass
Successful payment	Enter valid test card	Payment succeeds, order created	✓ Pass
Failed payment	Enter declined card	Error message displayed	✓ Pass
Order confirmation	After successful payment	Confirmation page shown	✓ Pass

5.2.5 Admin Dashboard

Test Case	Steps	Expected Result	Status
Create product	Fill add form, save	Product appears in table and shop	✓ Pass
Edit product	Modify fields, update	Changes reflected	✓ Pass
Delete product	Click delete, confirm	Product removed	✓ Pass
Update order status	Click ship/deliver	Status badge updates	✓ Pass
Reply to message	Type reply, send	Reply saved, status changes	✓ Pass

5.3 Responsive Testing

The application was tested on the following viewport sizes:

Device	Width	Status
Mobile	375px	✓ Renders correctly
Tablet	768px	✓ Renders correctly
Desktop	1280px	✓ Renders correctly
Large Desktop	1920px	✓ Renders correctly

5.4 Security Testing

Test Case	Result
SQL injection via form inputs	✔ Prevented by Supabase parameterized queries
Unauthorized admin access	✔ Blocked by RLS policies and client-side checks
JWT token validation	✔ Handled by Supabase Auth
Stripe payment secrets	✔ Processed server-side via Edge Function

5.5 Known Limitations

- **Image management:** Product images are stored in the public directory rather than Supabase Storage (future improvement).
- **Real-time updates:** The app uses polling rather than Supabase real-time subscriptions for simplicity.
- **No automated tests:** Unit and integration tests were not implemented due to time constraints.
- **No search functionality:** Products can only be browsed by category, not searched by keyword.

General Conclusion

This project successfully delivered a full-stack e-commerce platform for Maison Tislit, a brand specializing in Moroccan traditional feminine clothing. The application meets all primary objectives set at the outset of the project:

- A **responsive, elegant frontend** built with React and Tailwind CSS that reflects the brand's cultural identity.
- A **secure backend** powered by Supabase, providing authentication, a PostgreSQL database with Row Level Security, and serverless functions for payment processing.
- A **complete shopping experience** from product browsing and cart management to Stripe payment integration and order confirmation.
- A **comprehensive admin dashboard** enabling the brand to manage products, orders, customer messages, and users.
- **Multilingual support** with French and English interfaces.

Key Achievements

Area	Achievement
Frontend	10 pages, reusable component architecture, responsive design
Backend	5 database tables, RLS policies, migrations, Edge Functions
Auth	Full registration/login/logout with session management
Payments	Stripe integration with payment intents
Admin	5 management sections with CRUD operations
i18n	240+ translation keys across two languages

Future Perspectives

Several enhancements could be made in future iterations:

- Search functionality:** Implement full-text search across product names and descriptions.
- Product reviews:** Allow customers to leave ratings and reviews.
- Wishlist:** Create a dedicated favorites/wishlist feature.
- Email notifications:** Send order confirmation and shipping updates via email.
- Inventory management:** Real-time stock tracking with low-stock alerts.
- Analytics dashboard:** Detailed sales reports, customer insights, and conversion tracking.
- Mobile application:** Develop a React Native app for iOS and Android.
- Supabase Storage:** Migrate product images to Supabase Storage for better scalability.
- Automated testing:** Add unit tests (Vitest) and end-to-end tests (Cypress/Playwright).

Personal Reflection

Building Maison Tislit's e-commerce platform was a rewarding experience that combined frontend design, backend architecture, payment integration, and database management. The project demonstrated the power of modern web technologies — particularly the React + Supabase stack — in delivering a production-ready application with minimal infrastructure overhead. The result is a

platform that authentically represents Moroccan craftsmanship in the digital space while providing a seamless experience for customers across languages and devices.
